

Inteligencia Artificial

Informe Final: Problema Multi-Depot Vehicle Routing Problem with Time Windows (MDVRP-TW)

Domingo Ruiz-Tagle

23 de junio de 2026

Evaluación

Código Fuente (10 %):	_____
Resumen (2 %):	_____
Introducción (3 %):	_____
Representación (10 %):	_____
Descripción del algoritmo (20 %):	_____
Experimentos (10 %):	_____
Resultados (30 %):	_____
Conclusiones (10 %):	_____
Bibliografía (5 %):	_____
Nota Final (100):	_____

Resumen

El presente informe analiza el Problema de Ruteo de Vehículos Multi-Depósito con Ventanas de Tiempo (MDVRP-TW), un problema de optimización combinatoria clasificado como NP-Hard. El objetivo principal es minimizar los costos operativos y penalizaciones temporales considerando múltiples puntos de origen y plazos estrictos de atención. Se formula un modelo matemático de programación lineal mixta y se examina el estado del arte, evidenciando que, si bien los métodos exactos son ineficientes a escala industrial, las metaheurísticas híbridas actuales logran soluciones de alta calidad. Finalmente, se discute la transición hacia la logística verde (GVRP) como tendencia crítica para incorporar variables de sostenibilidad y autonomía energética.

1. Introducción

La logística moderna exige la optimización continua de las cadenas de suministro para reducir costos operativos y mejorar el nivel de servicio. El **Vehicle Routing Problem** (VRP) es el marco teórico central que modela estas decisiones: dado un conjunto de clientes con demandas conocidas y una flota de vehículos que parten de uno o más depósitos, el VRP busca determinar el conjunto de rutas de costo mínimo tal que cada cliente sea visitado exactamente una vez y se respeten las restricciones operativas de la flota. En este contexto, este informe tiene como propósito analizar el **MDVRP-TW** (Multi-Depot VRP with Time Windows), una variante del VRP que incorpora dos extensiones críticas: múltiples centros de distribución como puntos de partida y retorno de los vehículos, y ventanas de tiempo que acotan el intervalo horario válido para atender a cada cliente. El análisis abarca su formulación matemática como programa lineal

mixto, la evolución de los métodos de resolución desde enfoques exactos hasta metaheurísticas híbridas, y una experimentación comparativa de tres operadores de búsqueda local sobre instancias del estándar Solomon. La motivación de esta investigación nace de la creciente necesidad por sistemas de distribución más ágiles y sostenibles, para garantizar la rentabilidad de las operaciones de transporte actuales.

El informe se estructura de la siguiente manera: primero, se presenta una definición del problema, sus variables y restricciones fundamentales, junto con las variantes más relevantes que han derivado de él. Luego, se expone el estado del arte, evaluando la evolución cronológica de los métodos de resolución, sus ventajas, limitaciones y el paradigma híbrido dominante en la literatura reciente. Posteriormente, se plantea un modelo matemático de programación lineal mixta con sus conjuntos, parámetros, variables de decisión y restricciones operativas. Para finalizar, en las conclusiones se sintetizan las barreras tecnológicas que aún enfrenta el ruteo de vehículos y se perfilan los próximos pasos de la industria, fuertemente marcados por la necesidad de una logística sostenible.

2. Definición del Problema

MDVRP-TW, o mejor conocido como Multi-Depot Vehicle Routing Problem with Time Windows, es un problema proveniente de Vehicle Routing Problem (VRP), uno de los problemas de optimización combinatoria más estudiados desde su formulación original introducida por Dantzig y Ramser en 1959. En su forma básica busca encontrar una forma de asignar estaciones a camiones de tal manera que las demandas de las estaciones se satisfagan y el kilometraje total recorrido por la flota sea mínimo. Se proporciona un procedimiento basado en una formulación de programación lineal para obtener una solución cercana a la óptima [3].

A diferencia del modelo original, el MDVRP-TW añade dos capas de complejidad crítica para la logística moderna. Formalmente, el problema se define sobre un grafo dirigido completo $G = (V, A)$, donde $V = D \cup C$ es el conjunto de nodos con $D = \{d_1, \dots, d_m\}$ el conjunto de depósitos y $C = \{c_1, \dots, c_n\}$ el conjunto de clientes, y $A = \{(i, j) \mid i, j \in V, i \neq j\}$ el conjunto de arcos que representan los trayectos posibles. A cada arco (i, j) se le asocia una distancia $d_{ij} \geq 0$ y un tiempo de viaje $t_{ij} \geq 0$. Cada cliente $c_i \in C$ tiene una demanda $q_i > 0$ y una ventana de tiempo $[e_i, l_i]$ con tiempo de servicio s_i . Cada depósito $d_r \in D$ dispone de una flota de k vehículos homogéneos de capacidad Q . El objetivo es asignar clientes a depósitos y construir rutas para cada vehículo que minimicen el costo total, respetando las restricciones de capacidad, ventanas de tiempo y retorno al depósito de origen. Las dos extensiones respecto al VRP clásico son:

- **Múltiples puntos de origen:** Los vehículos no parten de una única terminal, sino de diversos depósitos, lo que requiere una decisión previa de asignación cliente-depósito.
- **Time Windows:** Cada cliente debe ser atendido dentro de un intervalo temporal definido, donde se establece un tiempo de llegada más temprano y el más tardío permitido para el servicio.

Computacionalmente, este problema pertenece a la clase **NP-Hard**, lo que implica que encontrar la solución óptima global requiere un costo computacional prohibitivo para instancias de gran escala. Además, combina características del problema de la mochila (restricción de capacidad) y del problema del vendedor viajero (TSP).

Para la formalización del MDVRP-TW, es necesario considerar un conjunto de componentes críticos que definen al problema:

- **Variables de decisión:** Se centran principalmente en la asignación de clientes a depósitos y la determinación de la secuencia de visitas (rutas) que minimicen el costo operativo.
- **Restricciones fundamentales:** Además de la capacidad de carga y las ventanas de tiempo ya mencionadas, se debe respetar:
 - **Visita única:** Cada cliente del conjunto total debe ser visitado exactamente una vez.
 - **Capacidad de flota:** La suma de las demandas de los clientes en una ruta no puede exceder el límite de carga del vehículo.

- **Continuidad temporal:** Todos los vehículos inician su operación simultáneamente al comienzo del tiempo programado.
- **Retorno a base:** Los vehículos deben comenzar y finalizar su trayecto en el mismo depósito de origen.
- **Lógica de servicio:** Si un vehículo llega antes del tiempo permitido, debe esperar. Si llega después, se aplica una penalización proporcional al tiempo de retraso incurrido.
- **Función objetivo:** El propósito principal es la minimización de una función de costo global, la cual suele integrar la distancia total recorrida, el número de vehículos activos y, en algunos casos, las penalizaciones por retrasos en las entregas.

MDVRP-TW, al ser una variante, representa solo una fracción de los problemas de VRP. Se presentan a continuación otras de las variantes más relevantes:

- **Heterogeneous VRP (HVRPTW):** A diferencia del modelo estándar que asume una flota uniforme, esta variante considera vehículos con distintas capacidades, costos operativos y autonomías. Introduce la necesidad de realizar una asignación estratégica del tipo de vehículo por ruta, lo que incrementa la complejidad combinatoria al añadir una dimensión de decisión tipo Bin Packing al ruteo [1].
- **Capacitated VRP (CVRP):** El volumen de pedidos despachados en una sola ruta está condicionado por la capacidad máxima del vehículo. Esta restricción asegura que la demanda agregada de los nodos visitados no exceda la disponibilidad de carga del transporte.
- **Dynamic VRP (DVRPTW):** Representa el escenario operativo más complejo, donde la información se revela de forma estocástica durante la ejecución de las rutas. Esta variante invalida las soluciones estáticas y exige el uso de algoritmos de re-optimización en tiempo real capaces de actualizar el plan de despacho en segundos [6].
- **Multi-Depot VRP (MDVRP):** El sistema no se limita a un punto de despacho centralizado, sino que dispone de diversos centros logísticos o depósitos desde los cuales la flota puede iniciar sus recorridos.
- **Green VRP with Time Windows (GVRPTW):** Esta variante integra objetivos de sostenibilidad ambiental mediante la optimización del consumo de energía o la reducción de emisiones de CO₂. A diferencia del modelo clásico, considera vehículos eléctricos o de combustible alternativo, incorporando restricciones adicionales como la autonomía limitada de las baterías y la necesidad de visitar estaciones de carga dentro de ventanas de tiempo específicas [7].

3. Estado del Arte

3.1. Hito Fundacional y Evolución Cronológica

El estudio sistemático del ruteo de vehículos se origina con Dantzig y Ramser [3], quienes en 1959 formularon el problema del despacho de camiones como un programa lineal, estableciendo la base formal sobre la cual se construirían décadas de investigación. Su contribución no fue únicamente técnica: definió el paradigma de modelar decisiones logísticas como problemas de optimización combinatoria, sentando el lenguaje matemático que aún domina la literatura.

Durante las siguientes décadas, la investigación avanzó hacia variantes progresivamente más realistas. El salto cualitativo más relevante para la industria llegó con Solomon [8] en 1987, quien introdujo las restricciones de ventanas de tiempo al VRP clásico, generando el estándar VRPTW. Su contribución fue doble: además de formalizar el problema, propuso un conjunto de instancias de *benchmark* (conjuntos C, R y RC) que se convirtieron en el estándar de evaluación comparativa de la comunidad durante más de tres décadas. Este hito es crítico porque permitió, por primera vez, comparar algoritmos de distintos autores sobre una base objetiva y reproducible.

La transición hacia el MDVRP-TW representa una complejidad adicional sustancial: al incorporar múltiples depósitos, el espacio de decisión se amplía con la asignación previa de clientes a depósitos, un subproblema que por sí solo es *NP-Hard*. Toth y Vigo [9] documentan cómo esta explosión combinatoria hace inviables los métodos exactos a escala industrial, marcando el inicio de la era de las heurísticas como enfoque dominante.

3.2. Métodos de Resolución: Ventajas, Limitaciones y Comparativa Crítica

A lo largo de la evolución del problema, tres familias de métodos han competido por establecerse como el enfoque dominante, cada una con *trade-offs* bien documentados.

- **Métodos Exactos:** Garantizan la optimalidad matemática mediante técnicas como *Branch-and-Bound* o programación entera mixta. Su ventaja es irrefutable en instancias pequeñas: la solución obtenida es, por definición, la mejor posible. Sin embargo, Toth y Vigo [9] demuestran empíricamente que su limitación es estructural e irresoluble: el tiempo de cómputo crece de forma exponencial con el tamaño de la instancia, haciéndolos inviables para problemas con más de 100 nodos. En la práctica logística, donde una red de distribución puede involucrar cientos o miles de clientes, esta familia de métodos queda relegada a la validación de soluciones en instancias reducidas o como cota inferior de referencia.
- **Heurísticas y Metaheurísticas:** Surgieron precisamente para superar esta barrera de escalabilidad. Bräysy y Gendreau [2] realizan una revisión sistemática de metaheurísticas aplicadas al VRPTW, concluyendo que algoritmos como Búsqueda Tabú, Recocido Simulado y los Algoritmos Genéticos (GA) logran soluciones de alta calidad en tiempos de cómputo acotados y polinomiales. No obstante, su análisis expone la limitación crítica compartida por esta familia: la sensibilidad paramétrica. El rendimiento de estos algoritmos depende fuertemente de la calibración de hiperparámetros, cuyo ajuste óptimo varía según la instancia. Zhang et al. [10] corroboran esta debilidad en el contexto específico de múltiples vehículos, confirmando que los GA presentan riesgo de convergencia prematura hacia óptimos locales cuando los parámetros no están correctamente ajustados.

La comparación entre ambas familias no es simplemente de velocidad versus calidad: es una elección entre certeza y viabilidad. Los métodos exactos ofrecen certeza pero son inviables; las metaheurísticas son viables pero sin garantía de optimalidad. Esta tensión es el motor que impulsa la hibridación como paradigma dominante en la literatura reciente.

3.3. Representaciones Híbridas: El Paradigma Dominante

La respuesta de la comunidad científica a las limitaciones individuales de cada familia ha sido la hibridación sistemática. Li et al. [4] presentan uno de los enfoques más representativos de esta tendencia: una estrategia en dos etapas que combina *K-means Clustering* con *Ant Colony Optimization* (ACO). En la primera etapa, el *Clustering* agrupa geográficamente a los clientes y los asigna a depósitos, reduciendo drásticamente el espacio de búsqueda al descomponer el problema global en subproblemas locales manejables. En la segunda etapa, ACO resuelve el ruteo dentro de cada clúster, aprovechando su mecanismo de estigmergia para explorar el espacio de soluciones de forma distribuida y adaptativa.

La fortaleza de este enfoque híbrido es precisamente que neutraliza las debilidades individuales de sus componentes: el *Clustering* controla la explosión combinatoria del multi-depósito, mientras que ACO aporta robustez ante óptimos locales. Comparado con GA puro [10], este esquema resulta más estable ante variaciones de instancia, ya que la descomposición espacial previa reduce la dependencia en la calibración de parámetros.

En el extremo de la complejidad, Baldacci et al. [1] introducen la dimensión de heterogeneidad de flota (HVRPTW), donde los vehículos difieren en capacidad y costo operativo. Esto añade al ruteo un subproblema de asignación tipo *Bin Packing*: no solo se decide el orden de visita, sino también qué tipo de vehículo realiza cada ruta. Este incremento en la dimensionalidad exige rediseñar los operadores de búsqueda, ya que una solución factible para flota homogénea puede ser infactible al considerar capacidades diferenciadas.

En términos de representación, el esquema dominante en la literatura reciente modela el MDVRP-TW sobre un grafo completo $G = (V, A)$, donde los depósitos y clientes constituyen el conjunto de nodos V y los arcos A representan los trayectos posibles entre ellos. Li et al. [4] adoptan esta representación e incorporan la duplicación de nodos de depósito para permitir que cada vehículo inicie y finalice en el mismo origen, lo que simplifica la formulación de las restricciones de retorno a base. Esta representación ha demostrado ser la más efectiva para integrar simultáneamente las restricciones de capacidad, ventanas de tiempo y multi-depósito, dado que permite expresar la totalidad de las condiciones operativas como restricciones lineales sobre variables binarias de arco.

3.4. La Frontera Actual: Dinamismo y Logística Verde

La frontera más activa de la investigación contemporánea se articula en torno a dos desafíos que los modelos estáticos y homogéneos no pueden resolver.

- **Dinamismo Operativo (DVRPTW):** Psaraftis et al. [6] documentan tres décadas de investigación, concluyendo que la información que se revela en tiempo real durante la ejecución de las rutas (nuevos pedidos, cancelaciones, tráfico) invalida cualquier solución estática. Los algoritmos de re-optimización deben actualizar el plan de despacho en segundos, lo que impone restricciones de tiempo de cómputo incompatibles con la mayoría de las metaheurísticas convencionales.
- **Sostenibilidad Energética (GVRP / EVRPTW):** Schneider et al. [7] formalizan el problema incorporando autonomía limitada de batería y estaciones de recarga como nodos adicionales en el grafo. Montoya et al. [5] profundizan esta línea al modelar funciones de carga no lineales, un aspecto crítico que los modelos anteriores simplificaban incorrectamente. Esta no-linealidad transforma el subproblema de *scheduling* de carga en un problema de control óptimo. La limitación crítica que persiste es que añadir estaciones y tiempos de espera multiplica el espacio de búsqueda, exigiendo que las metaheurísticas de vanguardia sean rediseñadas con operadores especializados para mantener la factibilidad energética.

4. Modelo Matemático

La formulación presentada a continuación se basa en el modelo matemático propuesto por Li et al. [4], el cual aborda el problema de ruteo multi-depósito integrando restricciones de capacidad y ventanas de tiempo suaves (soft time windows). Esta representación ha sido adaptada para el presente informe, manteniendo la coherencia con los parámetros técnicos del problema de estudio.

4.1. Conjuntos e Índices

El espacio de búsqueda se define sobre un grafo completo $G = (V, A)$, donde:

- $D = \{1, \dots, m\}$: Conjunto de depósitos.
- $C = \{m + 1, \dots, m + n\}$: Conjunto de clientes.
- $V = D \cup C$: Conjunto total de nodos.
- K : Conjunto de vehículos disponibles, asumidos con capacidad homogénea.
- $(i, j) \in A$: Arcos que representan el trayecto desde el nodo i al nodo j .

4.2. Parámetros

- Q : Capacidad máxima de carga de cada vehículo.
- q_i : Demanda de carga del cliente $i \in C$.
- d_{ij} : Distancia entre el nodo i y el nodo j .
- t_{ij} : Tiempo de viaje entre i y j .
- s_i : Tiempo de servicio en el cliente i .
- $[e_i, l_i]$: Ventana de tiempo (inicio y fin) para atender al cliente i .
- $P_i(T_{ik})$: Función de penalización por violación de la ventana de tiempo.

4.3. Variables de Decisión

- $x_{ijk} \in \{0, 1\}$: Vale 1 si el vehículo k viaja directamente del nodo i al j , y 0 en caso contrario.
- T_{ik} : Tiempo de llegada del vehículo k al nodo i .

4.4. Función Objetivo

El modelo busca minimizar una función de costo global que integra la operación y el cumplimiento logístico:

$$\min Z = \sum_{k \in K} \sum_{i \in V} \sum_{j \in V} d_{ij} x_{ijk} + \sum_{k \in K} \sum_{i \in C} P_i(T_{ik}) \quad (1)$$

Donde el primer término representa la distancia total recorrida y el segundo término las penalizaciones por desvíos en las ventanas de tiempo.

4.5. Restricciones

El modelo está sujeto a las siguientes condiciones operativas:

$$\sum_{k \in K} \sum_{i \in V} x_{ijk} = 1, \quad \forall j \in C \quad (2)$$

La restricción (2) asegura que cada cliente sea visitado exactamente una vez por un solo vehículo.

$$\sum_{i \in V} x_{ipk} - \sum_{j \in V} x_{pjk} = 0, \quad \forall p \in C, \forall k \in K \quad (3)$$

La ecuación (3) garantiza la conservación de flujo: el vehículo que entra a un nodo debe salir de él.

$$\sum_{i \in C} q_i \left(\sum_{j \in V} x_{ijk} \right) \leq Q, \quad \forall k \in K \quad (4)$$

La restricción (4) limita la carga total de cada ruta a la capacidad máxima Q del vehículo.

$$T_{ik} + s_i + t_{ij} - M(1 - x_{ijk}) \leq T_{jk}, \quad \forall i, j \in V, \forall k \in K \quad (5)$$

La ecuación (5) define la continuidad temporal de los trayectos, utilizando una constante M (Big-M) para asegurar la validez de los tiempos de llegada.

$$\sum_{i \in D} \sum_{j \in C} x_{ijk} = \sum_{i \in C} \sum_{j \in D} x_{ijk} \leq 1, \forall k \in K \quad (6)$$

Finalmente, la restricción (6) obliga a que los vehículos comiencen y terminen su recorrido en el mismo depósito.

5. Representación

5.1. Estructura Primaria de la Solución

Una solución del MDVRP-TW se representa en la implementación computacional como un mapa hash que asocia cada depósito a un conjunto de rutas:

$$\text{Solucion} : \text{id.depósito} \longrightarrow [\text{Ruta}_1, \text{Ruta}_2, \dots, \text{Ruta}_k]$$

donde cada Ruta_j es un vector de punteros a nodos clientes, ordenados estrictamente según la secuencia lógica de visita. En C++, esta topología se traduce de la siguiente manera:

```
typedef map<string, vector<vector<Node*>>> Solucion;
```

La clave del mapa es el identificador alfanumérico del depósito (por ejemplo, "D0"), y cada elemento del vector externo corresponde a un vehículo asignado a ese centro. Un vehículo sin clientes asignados se representa estructuralmente como un vector vacío; esto permite mantener la disponibilidad fija de k vehículos por depósito sin necesidad de instanciar o destruir estructuras de memoria en tiempo de ejecución. Además, al usarse `std::map` y no `std::unordered_map`, la iteración sobre los depósitos (por ejemplo, en `calcular_fo_total`) se realiza siempre en orden lexicográfico de su `id`, lo cual vuelve determinista cualquier recorrido completo de la solución a costa de un acceso $O(\log m)$ por clave en lugar de $O(1)$ amortizado.

Cada nodo de la red lógica se modela mediante la estructura `Node`, que centraliza los parámetros estáticos requeridos para la evaluación de factibilidad:

```
struct Node {
string id;
double x, y; // coordenadas espaciales
double demand; // carga requerida
double e_i, l_i; // ventana temporal admisible
double s_i; // tiempo de descarga/servicio
bool is_depot; // flag l\ogico de clasificaci\on
};
```

Los nodos se instancian una única vez en `leer_instancia` y residen en el *heap* durante toda la ejecución; nunca son copiados ni mutados, solo referenciados. Esto es relevante porque el protocolo experimental (Sección 7) clona la solución inicial tres veces (`sol_swap`, `sol_relocate`, `sol_2opt`) para que cada operador opere de forma independiente: como `Solucion` es un contenedor de **punteros**, clonarla copia únicamente las direcciones de memoria y no los datos de cada cliente, por lo que las tres copias son estructuralmente independientes (se pueden reordenar sin interferir entre sí) pero comparten los mismos objetos `Node` subyacentes como fuente de verdad.

Las **variables de dominio dinámicas** que determinan el estado de la solución son:

- **Secuencia posicional:** El orden de los punteros `Node*` dentro del vector interno define implícitamente la distancia de los arcos y la propagación del tiempo de viaje.
- **Asignación de flota:** Determina en qué sub-vector reside un cliente. La lógica de ruteo exige que cada `Node*` exista exactamente en una coordenada de la matriz bidimensional.
- **Acumulación de carga y tiempo:** Variables dinámicas calculadas secuencialmente durante la función de evaluación para detectar excesos respecto al umbral Q o tardanzas frente al límite l_i .

Las perturbaciones geométricas (movimientos de búsqueda local) operan manipulando únicamente el orden de los punteros en los vectores, manteniendo intacta la integridad de la estructura de datos matriz.

5.2. Representación Derivada: Evaluación y Penalizaciones

A diferencia de los componentes anteriores, que describen el *estado* de la solución, el sistema mantiene una segunda capa de representación que describe la *calidad* de ese estado. Esta capa se construye dinámicamente cada vez que se invoca `evaluar_ruta` y se materializa en dos estructuras auxiliares:

```
struct Penalty {
string id;
double llegada;
double e_i, l_i;
double tardanza;
double exceso;
bool is_capacity;
};
```

```

struct EvalResult {
double dist;
double tiempo;
double carga;
double tardanza_total;
double exceso_carga;
vector<Penalty> penalizaciones;
};

```

`EvalResult` es, en la práctica, una **vista resumen** de una ruta: condensa la distancia recorrida, el reloj de simulación acumulado, la carga total y dos magnitudes de infactibilidad (`tardanza_total` y `exceso_carga`) que alimentan directamente la Función Objetivo (ecuación 7). El vector `penalizaciones` conserva, además, un registro nodo a nodo de *qué* cliente falló y *por cuánto*, lo que permite trazabilidad: el archivo de salida (`escribir_solucion`) reutiliza esta misma estructura para reportar, al final de cada corrida, exactamente qué clientes quedaron con tardanza o qué ruta excedió la capacidad, sin necesidad de recalcular nada. En otras palabras, la representación de la solución (`Solucion`) es deliberadamente *estática y minimalista* (solo orden y asignación), mientras que toda la semántica de factibilidad se delega a una representación derivada que se recalcula bajo demanda en cada evaluación.

5.3. Ejemplo Ilustrativo

Para concretar la notación anterior, considérese una instancia mínima con dos depósitos (D0, D1) y dos vehículos por depósito ($k = 2$). Tras la construcción heurística, el mapa `Solucion` podría adoptar el siguiente estado:

```

"D0" -> [ [C3, C7, C1], [] ]
"D1" -> [ [C5], [C2, C9] ]

```

Aquí, el primer vehículo de D0 visita a los clientes C3, C7 y C1 en ese orden; el segundo vehículo de D0 no tiene clientes asignados (vector vacío, pero existente); y la flota de D1 distribuye su carga entre un vehículo con un solo cliente y otro con dos. Un movimiento `Swap` entre las posiciones 0 y 2 de la primera ruta de D0 transformaría la secuencia en [C1, C7, C3]; un `Relocate` podría mover C5 desde D1 hacia el vehículo vacío de D0 (esto corresponde, de hecho, al operador inter-ruta descrito en la Sección 6.2, ya que cruza la frontera de depósito).

5.4. Costos y Decisiones de Diseño de la Representación

La elección de un número fijo de k vehículos por depósito (en lugar de una lista de tamaño variable que crezca según demanda) simplifica el código de inserción y evita la gestión de memoria dinámica de rutas, pero introduce un costo de espacio: si un depósito recibe menos vehículos útiles que k , los vectores vacíos persisten y se recorren igualmente en cada llamada a `calcular_fo_total` (con un costo marginal, ya que un `continue` los descarta de inmediato, pero sin liberar la entrada del vector externo). En términos de memoria, la representación completa de una solución ocupa $O(m \cdot k + n)$ punteros, frente a los $O(n)$ que requeriría una representación de permutación plana con separadores de ruta; el costo adicional se justifica porque la estructura matricial hace explícita la pertenencia simultánea a un depósito y a un vehículo, evitando tener que parsear separadores cada vez que un operador necesita identificar la ruta de un nodo.

Finalmente, cabe notar que esta representación no almacena de forma persistente ni la distancia acumulada ni el tiempo de llegada a cada nodo: ambos se recomputan íntegramente, desde el depósito, cada vez que se invoca `evaluar_ruta` sobre una ruta. Esta decisión de diseño – mantener la representación de la solución libre de caché de evaluación – es la raíz del problema de escalabilidad que se discute en la Sección 6 y en las Conclusiones: cualquier operador de vecindad, sin importar cuán local sea el cambio que introduce, fuerza una reevaluación completa de la ruta afectada.

6. Descripción del Algoritmo

La resolución del problema se estructuró bajo un diseño en dos fases: una construcción heurística restrictiva que prioriza la factibilidad temporal, seguida de un proceso de explotación local intensiva (*Hill Climbing*) basado en gradientes de penalización.

6.1. Fase 1: Construcción Heurística Sensible al Tiempo y al Espacio

A diferencia de un enfoque voraz estándar que prioriza únicamente la compacidad espacial o la saturación de capacidad, la solución inicial se construye integrando la viabilidad de las ventanas de tiempo como criterio principal de inserción. El ordenamiento previo de los clientes no depende exclusivamente de l_i : se calcula un *score* compuesto que penaliza la lejanía al depósito más cercano,

$$\text{score}(c) = l_c - 0,5 \cdot \min_{d \in D} d_{cd}, \quad (7)$$

y los clientes se ordenan de forma ascendente según este valor. La intuición de diseño es doble: por un lado, los clientes con ventana de tiempo más restrictiva (l_c pequeño) deben rutearse primero para evitar que queden “atrapados” al final de una ruta ya saturada de tiempo; por otro, entre dos clientes con urgencia similar, se prioriza al que está más alejado de cualquier depósito, bajo la premisa de que los nodos lejanos tienen menos oportunidades de inserción factible y conviene fijarlos en la topología antes que los nodos cómodos, que casi siempre encontrarán espacio en alguna ruta posterior.

Una vez ordenada la lista, el algoritmo recorre los depósitos y, dentro de cada depósito, los vehículos, en el orden en que fueron declarados (no se evalúan todas las combinaciones posibles para escoger la mejor, sino que se adopta la primera que cumple ambas condiciones: *first-fit*, no *best-fit*). Para cada candidato (d, v) se exige:

- **Factibilidad de capacidad:** la carga acumulada del vehículo más la demanda del nuevo cliente no debe superar Q .
- **Factibilidad temporal estricta:** al insertar tentativamente al cliente al final de la ruta y reevaluarla completa con `evaluar_ruta`, la tardanza total de *toda* la ruta (no solo la del nuevo cliente) debe ser exactamente cero (`eval_test.tardanza_total == 0`).

Si ningún par (d, v) satisface ambas condiciones, el algoritmo recurre a un mecanismo de *fallback*: se evalúa la inserción del cliente en **todos** los vehículos con capacidad disponible (sin exigir factibilidad temporal) y se selecciona aquel que produce el menor valor de la Función Objetivo de ruta (ecuación 7), es decir, el que introduce la menor magnitud de tardanza ponderada. Esta es una diferencia sutil pero relevante: mientras la fase principal es estrictamente *first-fit* y conservadora, el *fallback* es *best-fit* y tolera infactibilidad transitoria, delegando su corrección a la Fase 2. Esta combinación asegura una topología inicial con alta proporción de nodos factibles, sin bloquear la construcción cuando ningún vehículo admite al cliente sin violación.

6.2. Fase 2: Hill Climbing con Best Improvement e Inter-Ruta

La optimización aplica un esquema de *Hill Climbing* (HC) determinista sobre la solución inicial, articulado en dos operadores complementarios que actúan en secuencia estricta: primero el operador **intra-ruta** hasta su convergencia, y luego el operador **inter-ruta** hasta su propia convergencia.

Hill Climbing intra-ruta. En cada iteración, el algoritmo recorre *todas* las rutas de *todos* los depósitos y, dentro de cada ruta con al menos dos clientes, evalúa exhaustivamente el conjunto de movimientos generables por el operador activo (Swap, Relocate o 2-Opt) sobre todos los pares de posiciones (i, j) admisibles. Para cada movimiento candidato se calcula el costo de la ruta resultante y se compara contra el costo de la ruta original mediante un delta local:

$$\Delta(i, j) = \text{FO}_{\text{ruta}}(\text{vec_ruta}) - \text{FO}_{\text{ruta}}(\text{ruta original}).$$

A diferencia de una búsqueda *First Improvement*, el algoritmo **no aplica el primer movimiento que mejora**. En su lugar, conserva el mínimo global de Δ entre absolutamente todas las rutas, depósitos y

posiciones evaluadas en esa iteración. Solo entonces ejecuta el movimiento con mayor reducción, aplicando una estrategia **Best Improvement** a nivel de solución completa.

Este ciclo se repite hasta cumplir uno de dos criterios de parada:

- Ninguna perturbación produce un $\Delta < -10^{-4}$.
- Se alcanza el límite estricto de 1000 iteraciones.

Es fundamental notar una asimetría estructural en la evaluación de los operadores intra-ruta:

- **RELOCATE**: Evalúa todas las posiciones de inserción $j \neq i$, incluyendo aquellas anteriores a i .
- **SWAP** y **2-OPT**: Solo consideran pares con $j > i$. Dado que intercambiar (i, j) es equivalente a (j, i) , y revertir el segmento $[i, j]$ es idéntico a hacerlo en sentido contrario, esta restricción evita evaluar movimientos simétricamente redundantes sin perder cobertura de la vecindad.

Relocate Inter-Ruta. Tras alcanzar la convergencia del Hill Climbing intra-ruta, se invoca repetidamente la función `relocate_inter_ruta` mediante un patrón `while`. A diferencia de los operadores descritos en la Sección 6.4, esta función evalúa en una sola pasada el impacto de mover clientes entre distintas rutas. Para cada cliente i , evalúa:

1. **Costo de remoción (g)**: La ganancia obtenida al extraer al cliente i de su ruta de origen (d_1, v_1) .

$$g = \text{FO}(\text{ruta}_1) - \text{FO}(\text{ruta}_1 \setminus \{i\})$$

2. **Costo de inserción (c)**: El costo de añadir al cliente i en cada posición j de cualquier otra ruta (d_2, v_2) del sistema, incluyendo depósitos distintos.

$$c = \text{FO}(\text{ruta}_2 \cup \{i\}) - \text{FO}(\text{ruta}_2)$$

El movimiento global se acepta estrictamente si representa una mejora, es decir, si $\delta = c - g < 0$. De todas las combinaciones $(i, d_1, v_1, j, d_2, v_2)$ evaluadas en la vecindad, se ejecuta únicamente la de menor δ . Si ninguna combinación logra mejorar la solución, la función retorna `false` y el ciclo principal se detiene.

En la práctica, este operador es el único mecanismo capaz de mover clientes **entre depósitos distintos**. Esto lo convierte en el verdadero motor para resolver la dimensión “multi-depósito” del problema, superando las limitaciones de los operadores intra-ruta, los cuales operan dentro de una única ruta y son ciegos a la asignación cliente-depósito.

6.3. Función de Evaluación con Gradiente Proporcional

Para evitar el estancamiento de la búsqueda local en zonas topológicas planas (común en esquemas de penalización binaria o multiplicadores fijos), se diseñó una función objetivo estructurada sobre penalizaciones proporcionales a la magnitud de la violación:

$$Z = \text{distancia_total} + 100,0 \cdot \sum \max(0, T_{ik} - l_i) + 500,0 \cdot \max(0, C_k - Q) \quad (8)$$

Este gradiente continuo permite que el algoritmo detecte y ejecute perturbaciones que, si bien no logran la factibilidad inmediata, reducen la magnitud del error logístico, guiando matemáticamente la búsqueda hacia la zona factible. La evaluación subyacente (`evaluar_ruta`) simula el avance del vehículo nodo a nodo: si la llegada ocurre antes de e_i , el reloj de la ruta se adelanta hasta e_i (espera sin costo explícito); si ocurre después de l_i , se registra la tardanza pero el reloj **no** se trunca a l_i , sino que continúa desde la hora real de llegada. Esta decisión es crítica para el comportamiento observado en los resultados: una vez que una ruta acumula retraso, dicho retraso se propaga y normalmente se agrava en los clientes subsiguientes, lo que penaliza fuertemente – y de forma realista – a las rutas mal secuenciadas. La verificación de capacidad, en cambio, se evalúa una sola vez al final del recorrido (carga total contra Q), no de forma incremental nodo a nodo; esto es coherente con el hecho de que la demanda de un cliente no depende del orden en que se visita, solo de su pertenencia a la ruta.

Los coeficientes 100,0 y 500,0 son constantes fijas (no calibradas empíricamente ni ajustadas por instancia) cuya única función es asegurar que ninguna mejora de distancia pura pueda compensar una unidad de tardanza o de exceso de capacidad, forzando al algoritmo a priorizar la factibilidad sobre la optimalidad geométrica mientras existan violaciones activas.

6.4. Operadores de Vecindad Intra-Ruta

El modelo implementa tres operadores topológicos, evaluados de manera independiente:

- **Swap** ($O(N^2)$): Intercambia la secuencia de visita de dos clientes. Efectivo para corregir inversiones locales de precedencia, pero conserva el resto de la secuencia intacta, por lo que su capacidad de explorar configuraciones topológicamente distantes es limitada.
- **Relocate** ($O(N^2)$): Extrae un nodo de su índice actual y lo reinserta en cualquier otra posición de la ruta. Su alto índice de movilidad (un solo movimiento puede desplazar un nodo a través de toda la secuencia) lo hace superior en la resolución de restricciones temporales complejas, donde basta reubicar un único cliente mal posicionado para eliminar una cadena de tardanzas.
- **2-Opt** ($O(N^2)$): Invierte el subsegmento comprendido entre dos nodos. Fundamental para eliminar cruces geométricos en instancias con alta densidad espacial, aunque al revertir un segmento completo invierte también el orden temporal relativo de todos los clientes intermedios, lo que puede introducir nuevas violaciones de ventana de tiempo si el segmento no era simétrico en sus urgencias.

Cada evaluación de movimiento, independientemente del operador seleccionado, exige clonar la ruta completa (`vector<Node*>vec_ruta = ruta;`) y recalcular desde cero su distancia y tiempo respecto al depósito mediante la función `evaluar_ruta`. Este enfoque ingenuo define la complejidad del proceso:

- **Costo por evaluación:** $O(N)$, debido a la necesidad de recorrer todos los nodos de la ruta para recalcular la función objetivo.
- **Espacio de búsqueda:** $O(N^2)$, correspondiente a la cantidad de pares (i, j) evaluables por cada ruta.
- **Costo por iteración:** $O(N^3)$, resultante de explorar la vecindad completa de **una** sola ruta en un paso del Hill Climbing.

Al sumar este costo sobre todas las rutas y depósitos, y repetirlo hasta alcanzar la convergencia, se genera un evidente **cuello de botella algorítmico**. Este impacto crítico en el rendimiento se cuantifica en la Sección 8 y se discute como la principal limitación estructural en las Conclusiones.

7. Experimentos

7.1. Instancias de Validación

Los ensayos computacionales se ejecutaron sobre un subconjunto truncado a 25 nodos de las instancias estándar de Solomon [8] (que en su versión original contemplan 100 clientes). Esta reducción dimensional permite aislar el comportamiento lógico de los operadores frente a distintas topografías, minimizando el ruido introducido por la profundidad combinatoria, a costa de sacrificar representatividad respecto al desempeño a escala completa (limitación que se retoma en las Conclusiones).

- **C101 (Clustered):** 25 clientes, 3 vehículos, $Q = 200$. Nodos estrictamente agrupados con ventanas de tiempo rígidas.
- **R201 (Random):** 25 clientes, 1 vehículo, $Q = 1000$. Distribución espacial aleatoria con ventanas de tiempo holgadas, lo que maximiza la longitud de las rutas.
- **RC101 (Random-Clustered):** 25 clientes, 3 vehículos, $Q = 200$. Escenario híbrido que combina núcleos de alta densidad con entregas aisladas.

Cada instancia fue preprocesada a un formato de texto plano propio, leído por la función `leer_instancia`, que separa explícitamente la cabecera de configuración (número de depósitos, vehículos por depósito y capacidad), el bloque `DEPOSITOS` y el bloque `CLIENTES`:

```

DEPOSITOS: <m> VEHICULOS_POR_DEPOSITO: <k> CAPACIDAD: <Q>
DEPOSITOS
id x y demand e_i l_i s_i
...
CLIENTES
id x y demand e_i l_i s_i
...

```

Es relevante notar que el número de vehículos por depósito (k) se lee como un único valor global aplicado a todos los depósitos de la instancia, y no varía entre depósitos; esto simplifica el parser, pero implica que el diseño actual no admite flotas heterogéneas entre centros de distribución sin modificar el formato de entrada. En el caso de R201, el valor reportado de un único vehículo es, en rigor, un único vehículo *por cada* depósito definido en la instancia; dado que la instancia se trunca y se simplifica a un escenario predominantemente mono-depósito para esta prueba, el efecto neto observado es el de un solo vehículo disponible en el sistema, lo cual se analiza en profundidad en la Sección 8.

7.2. Diseño Experimental

El experimento se diseñó como una comparación *intra-instancia* de tres tratamientos (los operadores Swap, Relocate y 2-Opt), controlando la solución de partida para que sea idéntica en los tres casos. Las variables del diseño son:

- **Variable independiente:** el operador de vecindad intra-ruta utilizado durante la Fase 2 (Swap, Relocate o 2-Opt).
- **Variable de bloqueo:** la instancia (C101, R201, RC101), cuya topología geométrica y rigidez temporal son fijas y conocidas, lo que permite atribuir diferencias de desempeño al operador y no a variaciones del problema.
- **Variables dependientes:** el valor final de la Función Objetivo, la distancia total recorrida, el tiempo de cómputo (en segundos, medido con `chrono::high_resolution_clock`) y el número de violaciones de restricciones (tardanza y exceso de capacidad) reportadas en `EvalResult`.

Dada la naturaleza determinista del *Hill Climbing* implementado – no existe componente aleatorio alguno en la construcción greedy ni en la selección de movimientos, que siempre es *Best Improvement* sobre el universo completo de candidatos –, no se requiere experimentación estocástica, múltiples corridas con distintas semillas, ni calibración de hiperparámetros probabilísticos: una sola ejecución por combinación (instancia, operador) es suficiente para caracterizar completamente su comportamiento. El límite máximo de iteraciones del HC intra-ruta se estableció en 1000 únicamente como salvaguarda contra ciclos infinitos por errores numéricos de punto flotante, no como un parámetro de calibración activo: en la práctica, ninguna de las nueve corridas (3 instancias $\times$ 3 operadores) se acercó a dicho límite, ya que la convergencia ocurre naturalmente en un número de iteraciones muy inferior dado el tamaño reducido de las instancias.

7.3. Protocolo Experimental

El protocolo consistió en las siguientes etapas secuenciales:

1. Generar una solución constructiva única por instancia mediante la función `generar_solucion_inicial_greedy`.
2. Clonar dicha solución tres veces para garantizar puntos de partida idénticos en las optimizaciones posteriores.
3. Someter cada clon, de forma independiente, a la función `optimizar_hc` con el operador correspondiente (Swap, Relocate o 2-Opt) hasta alcanzar su convergencia intra-ruta.
4. Ejecutar el ciclo de mejora `while (relocate_inter_ruta(...))` sobre cada clon, iterando hasta que no existan movimientos inter-ruta que favorezcan la función objetivo.

5. Registrar el valor final de la FO, la distancia total, las violaciones acumuladas y el tiempo de cómputo. Finalmente, exportar el detalle completo de cada ruta y sus penalizaciones a un archivo de texto independiente (`outputs/salida_{operador}_{instancia}.txt`).

8. Resultados

8.1. Síntesis Numérica

Las Tablas 1, 2 y 3 exponen el rendimiento terminal de los algoritmos de búsqueda local. Es imperativo destacar que, para las tres instancias y bajo todos los operadores, **la Función Objetivo igualó de forma exacta a la Distancia Total**, confirmando que las penalizaciones fueron reducidas a cero y evidenciando un 100 % de factibilidad operativa.

Cuadro 1: Resultados instancia C101 (25 nodos, FO Inicial: 500.73)

Operador	FO Final	Distancia	Tiempo Cómput. (s)	Violaciones
Swap	198.82	198.82	0.00	0
Relocate	198.82	198.82	0.00	0
2-Opt	198.82	198.82	0.00	0

Cuadro 2: Resultados instancia R201 (25 nodos, FO Inicial: 848.22)

Operador	FO Final	Distancia	Tiempo Cómput. (s)	Violaciones
Swap	575.64	575.64	0.00	0
Relocate	495.94	495.94	0.00	0
2-Opt	575.64	575.64	0.00	0

Cuadro 3: Resultados instancia RC101 (25 nodos, FO Inicial: 651.70)

Operador	FO Final	Distancia	Tiempo Cómput. (s)	Violaciones
Swap	475.33	475.33	0.00	0
Relocate	475.33	475.33	0.00	0
2-Opt	475.33	475.33	0.00	0

8.2. Análisis Técnico

El Efecto Embudo en C101 y RC101: Los resultados evidencian una convergencia idéntica (198.82 y 475.33 respectivamente) independientemente del operador inicial utilizado. Este comportamiento es resultado del diseño arquitectónico del *main*, donde la instrucción `while (relocate_inter_ruta(...))` actúa como un mecanismo post-procesador. Las topologías *Clustered* poseen ventanas de tiempo altamente rígidas, las cuales obligan a los operadores intra-ruta a detenerse en óptimos locales tempranos. Posteriormente, el algoritmo de extracción inter-ruta arrastra las tres configuraciones (previamente optimizadas por Swap, Relocate y 2-Opt) hacia el mismo "sumidero" geométrico, anulando las variaciones preliminares.

Superioridad del Operador en R201: La instancia de distribución puramente aleatoria (R201) es la única que permitió evadir el efecto embudo. Al operar con un único vehículo ($Q = 1000$), la heurística inter-ruta queda invalidada, exponiendo el rendimiento real de la mecánica intra-ruta. Aquí, Relocate demostró una superioridad aplastante, comprimiendo la distancia a 495.94 unidades, frente a los 575.64 de Swap y 2-Opt. En una distribución entrópica con ventanas holgadas, el intercambio estático de posiciones vecinas es

ineficaz; se requiere la flexibilidad del Relocate para extirpar nodos de raíz e inyectarlos en secuencias lógicas distintas.

Tiempos de Cómputo y Complejidad: La ejecución se resolvió de manera instantánea ($\leq 0,00s$) en todos los frentes. Este fenómeno se explica por el tamaño reducido de las instancias ($N = 25$) acoplado al nivel máximo de optimización en la compilación (-O3 en g++). Sin embargo, bajo el capó analítico, el diseño posee una fuerte dependencia a la fuerza bruta del procesador que enmascara la complejidad temporal real de las evaluaciones.

9. Conclusiones

La resolución del MDVRP-TW a través de una arquitectura *Hill Climbing* de dos fases logró un nivel de eficiencia empírica indiscutible sobre las instancias evaluadas. La transición desde un esquema de penalizaciones estático hacia una Función Objetivo basada en el castigo proporcional de la magnitud de la tardanza probó ser la clave matemática del modelo, otorgándole al algoritmo un gradiente continuo indispensable para navegar un espacio de búsqueda fuertemente restringido. Este diseño logró reducir la infactibilidad temporal a cero en todas las simulaciones operativas.

El análisis topológico demuestra que el comportamiento geométrico de las redes logísticas dicta el rendimiento del operador heurístico. En ecosistemas aleatorios con amplios márgenes de operación, las técnicas de reubicación agresiva (Relocate) dominan la optimización de trayectorias. Por el contrario, en escenarios aglomerados, la rigidez de las ventanas temporales provoca que la búsqueda local sea absorbida rápidamente por sumideros locales, requiriendo operadores inter-ruta como única vía de escape topológica.

Desde un punto de vista crítico y analítico, es imperativo señalar que el éxito operativo en tiempos de $\approx 0,00s$ es consecuencia directa de la reducción dimensional del grafo ($N = 25$) y de las capacidades del hardware actual. Actualmente, cada perturbación clona la ruta en memoria y recalcula las distancias desde el depósito en $O(N)$, elevando la exploración del vecindario a un costo técnico de $O(N^3)$. A nivel de escala industrial, con redes logísticas de 500 o 1000 nodos, este enfoque es computacionalmente inviable.

En conclusión, si bien el modelo desarrollado cumple exitosamente con los requisitos funcionales del VRPTW, el próximo hito tecnológico para garantizar la escalabilidad industrial debe ser obligatoriamente la transición matemática hacia evaluaciones de impacto local en tiempo constante $O(1)$.

10. Uso LLMs

1. Herramientas utilizadas:

- Gemini 2.0 Flash (plan Pro).
- Claude Sonnet.

2. Propósito de uso:

- Búsqueda y localización de bibliografía.
- Traducción y comprensión de abstracts en inglés.
- Corrección de redacción y estilo académico.
- Asistencia en la escritura de expresiones matemáticas en LaTeX.
- Apoyo en la implementación de partes del código y depuración de errores.
- Apoyo en el análisis e interpretación de resultados experimentales.
- Resolución de problemas relacionados con compilación y formato del informe.

3. **Intervención:** Las herramientas de IA fueron utilizadas como apoyo para la búsqueda bibliográfica, corrección de redacción, asistencia en LaTeX y comprensión de artículos científicos. Además, se utilizaron para discutir alternativas de implementación, corregir errores de programación y obtener retroalimentación sobre el análisis de los resultados experimentales. La integración final del código, la ejecución de los experimentos y la elaboración de las conclusiones fueron realizadas por el estudiante.

4. **Validación:** Toda la información entregada por la IA fue contrastada directamente con la base de datos de *IEEE Xplore* y la *Biblioteca Digital UTFSM*. Las fórmulas y conceptos NP-Hard fueron validados con los apuntes de la asignatura y la bibliografía oficial del curso.

Referencias

- [1] Roberto Baldacci, Maria Battarra, and Daniele Vigo. Routing a heterogeneous fleet of vehicles. *The Vehicle Routing Problem*, pages 3–27, 2008.
- [2] Olli Bräysy and Michel Gendreau. Vehicle routing problem with time windows, part ii: Metaheuristics. *Transportation Science*, 39(1):119–139, 2005.
- [3] George B. Dantzig and John H. Ramser. The truck dispatching problem. *Management Science*, 6(1):80–91, 1959.
- [4] Xiaohui Li, Bolong He, Yi Zhao, and Yuan Dong. A two-stage optimization approach for multi-depot vehicle routing problem with time windows. In *2024 4th International Conference on Computer Communication and Artificial Intelligence (CCAI)*, pages 540–545, 2024.
- [5] Alejandro Montoya, Christelle Guéret, Jorge E. Mendoza, and Juan G. Villegas. The electric vehicle routing problem with nonlinear charging function. *Transportation Research Part B: Methodological*, 103:87–110, 2017.
- [6] Harilaos N. Psaraftis, Min Wen, and Christos A. Kontovas. Dynamic vehicle routing problems: Three decades and counting. *Networks*, 67(1):3–31, 2016.
- [7] Michael Schneider, Andreas Stenger, and Dominik Goeke. The electric vehicle-routing problem with time windows and recharging stations. *Transportation Science*, 48(4):500–520, 2014.
- [8] Marius M. Solomon. Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2):254–265, 1987.
- [9] Paolo Toth and Daniele Vigo. An overview of vehicle routing problems. *The Vehicle Routing Problem*, pages 1–26, 2002.
- [10] Fangzhi Zhang, Yue Xie, and Liming Zheng. Application and efficiency analysis of genetic algorithm in multi-vehicle path optimisation models under time window constraints. In *2024 IEEE 7th International Conference on Information Systems and Computer Aided Education (ICISCAE)*, pages 938–943, 2024.